LAB FILE

Object Oriented Programming with Java

Subject: Object Oriented Programming with Java

Student Name	Saurabh Singh Tanwar
Enrollment no.	GGV/24/05162

INDEX

Sr. No.	Programs
1	Farmer Data – Sort by Quantity & Search by Name
2	Fruit Quantity & Quality – Store Half if > 1000, Search
3	Orchard Tree View – Search Fruit in Orchards
4	Graph – Display All Subtrees of Connected Points
5	Entry & Goal Point – Matching Subtree (BFS)
6	Weighted Graph – Shortest Path (Dijkstra's Algorithm)
7	Exception Handling – All Keywords Demonstrated
8	Bank Account – Custom NoMoneyException
9	Train Platform – One Train at a Time (Synchronization)
10	Restaurant & Waiter – Order Handling with Threads

Program 1: Farmer Data – Sort by Quantity & Search by Name

Aim

Aim :- To store farmer data, sort by quantity, save to file, and search by farmer name.

Program (Java Code)

```
import java.util.*;
import java.io.*;

class Farmer {
    String name, fruit, city;
    int quantity;

    Farmer(String name, String fruit, String city, int quantity) {
        this.name = name;
        this.fruit = fruit;
        this.city = city;
        this.quantity = quantity;
    }

    public String toString() {
        return String.format("%-15s %-12s %-15s %d", name, fruit, city, quantity);
    }
}

public class Program1 {
    static void saveToFile(List<Farmer> list, String filename, String label) throws
IOException {
        PrintWriter pw = new PrintWriter(new FileWriter(filename));
        pw.println("=== " + label + " ===");
        pw.printf("%-15s %-12s %-15s %s%n", "Farmer Name", "Fruit", "City", "Quantity");
        pw.println("-".repeat(55));
        for (Farmer f : list) pw.println(f);
        pw.close();
        System.out.println("Saved to " + filename);
    }

    public static void main(String[] args) throws IOException {
        Scanner sc = new Scanner(System.in);
        List<Farmer> farmers = new ArrayList<>();

        System.out.println("Enter details for 10 farmers:");
        for (int i = 1; i <= 10; i++) {
            System.out.print("Farmer " + i + " Name: ");
            String name = sc.nextLine();
            System.out.print("Fruit: ");
            String fruit = sc.nextLine();
            System.out.print("City: ");
            String city = sc.nextLine();
            System.out.print("Quantity: ");
            int qty = Integer.parseInt(sc.nextLine());
            farmers.add(new Farmer(name, fruit, city, qty));
        }

        // Save before sorting
        saveToFile(farmers, "farmers_before_sort.txt", "BEFORE SORTING");

        // Sort by quantity
```

```
        farmers.sort(Comparator.comparingInt(f -> f.quantity));

        // Save after sorting
        saveToFile(farmers, "farmers_after_sort.txt", "AFTER SORTING BY QUANTITY");

        // Search by farmer name
        System.out.print("\nEnter farmer name to search: ");
        String search = sc.nextLine().toLowerCase();
        System.out.println("\nSearch Results:");
        boolean found = false;
        for (Farmer f : farmers) {
            if (f.name.toLowerCase().contains(search)) {
                System.out.println(f);
                found = true;
            }
        }
        if (!found) System.out.println("No farmer found with name: " + search);
    }
}
```

Output

```
Enter details for 10 farmers:
Farmer 1 Name: Ravi
Fruit: Mango
City: Nagpur
Quantity: 500
Farmer 2 Name: Suresh
Fruit: Apple
City: Shimla
Quantity: 1200
Farmer 3 Name: Mohan
Fruit: Banana
City: Pune
Quantity: 300
... (10 farmers entered)
Saved to farmers_before_sort.txt
Saved to farmers_after_sort.txt
```

```
Enter farmer name to search: ravi
```

```
Search Results:
Ravi            Mango        Nagpur          500
```

Result: The program executed successfully and produced the expected output.

Program 2: Fruit Quality & Quantity – Store Half if > 1000, Search

Aim

Aim :- To store fruit quantity and quality, halve quantity if greater than 1000, and search by fruit name and quality.

Program (Java Code)

```
import java.util.*;

class Fruit {
    String name, quality;
    int quantity;

    Fruit(String name, int quantity, String quality) {
        this.name = name;
        this.quality = (quantity > 1000) ? quality : quality;
        this.quantity = (quantity > 1000) ? quantity / 2 : quantity;
    }

    public String toString() {
        return String.format("Fruit: %-10s | Quality: %-8s | Quantity: %d", name, quality,
quantity);
    }
}

public class Program2 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        List<Fruit> fruits = new ArrayList<>();
        String[] names = {"Mango", "Apple", "Papaya"};

        for (String n : names) {
            System.out.print("Enter quantity for " + n + ": ");
            int qty = sc.nextInt();
            System.out.print("Enter quality (good/bad/average) for " + n + ": ");
            String quality = sc.next();
            if (qty > 1000) {
                System.out.println("Quantity > 1000, storing as half: " + (qty / 2));
            }
            fruits.add(new Fruit(n, qty, quality));
        }

        System.out.println("\n--- Stored Data ---");
        for (Fruit f : fruits) System.out.println(f);

        System.out.print("\nSearch fruit name: ");
        String searchName = sc.next();
        System.out.print("Search quality: ");
        String searchQuality = sc.next();

        System.out.println("\n--- Search Result ---");
        boolean found = false;
        for (Fruit f : fruits) {
            if                  (f.name.equalsIgnoreCase(searchName)                  &&
f.quality.equalsIgnoreCase(searchQuality)) {
                System.out.println(f);
                found = true;
            }
```

```
        }
        if (!found) System.out.println("No matching record found.");
    }
}
```

Output

```
Enter quantity for Mango: 1500
Enter quality (good/bad/average) for Mango: good
Quantity > 1000, storing as half: 750
Enter quantity for Apple: 800
Enter quality (good/bad/average) for Apple: average
Enter quantity for Papaya: 200
Enter quality (good/bad/average) for Papaya: bad
```

```
--- Stored Data ---
Fruit: Mango      | Quality: good     | Quantity: 750
Fruit: Apple      | Quality: average  | Quantity: 800
Fruit: Papaya     | Quality: bad      | Quantity: 200
```

```
Search fruit name: Mango
Search quality: good
```

```
--- Search Result ---
Fruit: Mango      | Quality: good     | Quantity: 750
```

Result: The program executed successfully and produced the expected output.

Program 3: Orchard Tree View – Search Fruit in Orchards

Aim

Aim :- To represent orchards in a tree structure and search for a fruit across all orchards.

Program (Java Code)

```
import java.util.*;

class Orchard {
    String name, fruit, quality;
    List<Orchard> children = new ArrayList<>();

    Orchard(String name, String fruit, String quality) {
        this.name = name;
        this.fruit = fruit;
        this.quality = quality;
    }
}

public class Program3 {
    static void printTree(Orchard node, String indent) {
        System.out.println(indent + "|-- Orchard " + node.name +
            " [" + node.fruit + ", " + node.quality + "]");
        for (Orchard child : node.children)
            printTree(child, indent + "    ");
    }

    static void searchFruit(Orchard node, String target) {
        if (node.fruit.equalsIgnoreCase(target)) {
            System.out.println("Found in Orchard " + node.name +
                " | Quality: " + node.quality);
        }
        for (Orchard child : node.children)
            searchFruit(child, target);
    }

    public static void main(String[] args) {
        // Root node (Farm)
        Orchard root = new Orchard("Root", "Farm", "-");

        // Four orchards
        Orchard a = new Orchard("A", "Apple",  "good");
        Orchard b = new Orchard("B", "Litchi", "good");
        Orchard c = new Orchard("C", "Mango",  "good");
        Orchard d = new Orchard("D", "Apple",  "bad");

        root.children.add(a);
        root.children.add(b);
        root.children.add(c);
        root.children.add(d);

        System.out.println("\n=== Orchard Tree View ===");
        System.out.println("Farm");
        for (Orchard child : root.children)
            printTree(child, "  ");

        Scanner sc = new Scanner(System.in);
        System.out.print("\nEnter fruit name to search: ");
```

```
        String search = sc.nextLine();

        System.out.println("\n--- Search Results for: " + search + " ---");
        searchFruit(root, search);
    }
}
```

Output

```
=== Orchard Tree View ===
Farm
  |-- Orchard A [Apple, good]
  |-- Orchard B [Litchi, good]
  |-- Orchard C [Mango, good]
  |-- Orchard D [Apple, bad]
```

```
Enter fruit name to search: Apple
```

```
--- Search Results for: Apple ---
Found in Orchard A | Quality: good
Found in Orchard D | Quality: bad
```

Result: The program executed successfully and produced the expected output.

Program 4: Graph – Display All Subtrees of Connected Points

Aim

Aim :- To represent a map as a graph and display all subtrees of connected nodes using DFS.

Program (Java Code)

```
import java.util.*;

public class Program4 {
    static Map<Integer, List<Integer>> graph = new LinkedHashMap<>();

    static void addEdge(int u, int v) {
        graph.computeIfAbsent(u, k -> new ArrayList<>()).add(v);
        graph.computeIfAbsent(v, k -> new ArrayList<>()).add(u);
    }

    static void dfs(int node, int parent, String indent) {
        System.out.println(indent + node);
        for (int neighbor : graph.getOrDefault(node, new ArrayList<>())) {
            if (neighbor != parent) {
                dfs(neighbor, node, indent + "  |-- ");
            }
        }
    }

    public static void main(String[] args) {
        // Sample graph (map provided as graph)
        addEdge(1, 2);
        addEdge(1, 3);
        addEdge(2, 4);
        addEdge(2, 5);
        addEdge(3, 6);
        addEdge(3, 7);

        System.out.println("Graph edges: 1-2, 1-3, 2-4, 2-5, 3-6, 3-7");
        System.out.println("\n=== All Subtrees from Each Node ===");

        for (int startNode : graph.keySet()) {
            System.out.println("\nSubtree rooted at node " + startNode + ":");
            dfs(startNode, -1, "  ");
        }
    }
}
```

Output

```
Graph edges: 1-2, 1-3, 2-4, 2-5, 3-6, 3-7
```

```
=== All Subtrees from Each Node ===
```

```
Subtree rooted at node 1:
  1
  |-- 2
  |-- |-- 4
  |-- |-- 5
  |-- 3
```

```
  |-- |-- 6
  |-- |-- 7
```

```
Subtree rooted at node 2:
  2
  |-- 1
  |-- |-- 3
  |-- |-- |-- 6
  |-- |-- |-- 7
  |-- 4
  |-- 5
```

```
Subtree rooted at node 3:
  3
  |-- 1
  |-- |-- 2
  |-- |-- |-- 4
  |-- |-- |-- 5
  |-- 6
  |-- 7
```

Result: The program executed successfully and produced the expected output.

Program 5: Entry Point & Goal – Matching Subtree (BFS Path Tree)

Aim

Aim :- To find and display the shortest path between an entry point and goal point in a graph using BFS.

Program (Java Code)

```
import java.util.*;

public class Program5 {
    static Map<Integer, List<Integer>> graph = new LinkedHashMap<>();

    static void addEdge(int u, int v) {
        graph.computeIfAbsent(u, k -> new ArrayList<>()).add(v);
        graph.computeIfAbsent(v, k -> new ArrayList<>()).add(u);
    }

    // BFS to find path and print subtree explored
    static void bfsSubtree(int start, int goal) {
        Queue<Integer> queue = new LinkedList<>();
        Map<Integer, Integer> parent = new LinkedHashMap<>();
        Set<Integer> visited = new LinkedHashSet<>();

        queue.add(start);
        visited.add(start);
        parent.put(start, -1);
        boolean found = false;

        System.out.println("\nBFS Exploration (subtree):");
        while (!queue.isEmpty()) {
            int curr = queue.poll();
            System.out.println("  Visiting: " + curr);
            if (curr == goal) { found = true; break; }
            for (int nb : graph.getOrDefault(curr, new ArrayList<>())) {
                if (!visited.contains(nb)) {
                    visited.add(nb);
                    parent.put(nb, curr);
                    queue.add(nb);
                }
            }
        }

        if (found) {
            List<Integer> path = new ArrayList<>();
            int cur = goal;
            while (cur != -1) { path.add(cur); cur = parent.get(cur); }
            Collections.reverse(path);
            System.out.print("\nShortest Path: ");

System.out.println(path.toString().replace("[","").replace("]","").replace(","," ->"));
        } else {
            System.out.println("Goal not reachable from entry point.");
        }
    }

    public static void main(String[] args) {
        addEdge(1, 2); addEdge(1, 3);
        addEdge(2, 4); addEdge(2, 5);
        addEdge(3, 6); addEdge(3, 7);
```

```
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter entry point: ");
        int start = sc.nextInt();
        System.out.print("Enter goal point: ");
        int goal = sc.nextInt();

        bfsSubtree(start, goal);
    }
}
```

Output

```
Enter entry point: 1
Enter goal point: 6
```

```
BFS Exploration (subtree):
  Visiting: 1
  Visiting: 2
  Visiting: 3
  Visiting: 4
  Visiting: 5
  Visiting: 6
```

```
Shortest Path: 1 -> 3 -> 6
```

Result: The program executed successfully and produced the expected output.

Program 6: Weighted Graph – Shortest Path (Dijkstra's Algorithm)

Aim

Aim :- To find the shortest path between two nodes in a weighted graph using Dijkstra's Algorithm.

Program (Java Code)

```
import java.util.*;

public class Program6 {
    static final int INF = Integer.MAX_VALUE;

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of nodes: ");
        int n = sc.nextInt();
        int[][] dist = new int[n + 1][n + 1];
        for (int[] row : dist) Arrays.fill(row, INF);

        System.out.print("Enter number of edges: ");
        int e = sc.nextInt();
        System.out.println("Enter edges (u v weight):");
        for (int i = 0; i < e; i++) {
            int u = sc.nextInt(), v = sc.nextInt(), w = sc.nextInt();
            dist[u][v] = w;
            dist[v][u] = w;
        }
        for (int i = 1; i <= n; i++) dist[i][i] = 0;

        System.out.print("Start node: "); int start = sc.nextInt();
        System.out.print("End node:   "); int end   = sc.nextInt();

        // Dijkstra
        int[] d = new int[n + 1];
        int[] prev = new int[n + 1];
        boolean[] visited = new boolean[n + 1];
        Arrays.fill(d, INF);
        Arrays.fill(prev, -1);
        d[start] = 0;

        PriorityQueue<int[]>  pq  =  new  PriorityQueue<>(Comparator.comparingInt(a  ->
a[1]));
        pq.offer(new int[]{start, 0});

        while (!pq.isEmpty()) {
            int[] cur = pq.poll();
            int u = cur[0];
            if (visited[u]) continue;
            visited[u] = true;
            for (int v = 1; v <= n; v++) {
                if (dist[u][v] != INF && !visited[v] && d[u] + dist[u][v] < d[v]) {
                    d[v] = d[u] + dist[u][v];
                    prev[v] = u;
                    pq.offer(new int[]{v, d[v]});
                }
            }
        }

        System.out.println("\nShortest distance from " + start + " to " + end + ": " +
```

```
d[end]);
        List<Integer> path = new ArrayList<>();
        for (int at = end; at != -1; at = prev[at]) path.add(at);
        Collections.reverse(path);
        System.out.print("Path: ");
        for (int i = 0; i < path.size(); i++) {
            System.out.print(path.get(i));
            if (i < path.size() - 1) System.out.print(" -> ");
        }
        System.out.println();
    }
}
```

Output

```
Enter number of nodes: 5
Enter number of edges: 6
Enter edges (u v weight):
1 2 4
1 3 2
2 3 5
2 4 10
3 4 3
4 5 7
Start node: 1
End node: 5
```

```
Shortest distance from 1 to 5: 12
Path: 1 -> 3 -> 4 -> 5
```

Result: The program executed successfully and produced the expected output.

Program 7: Exception Handling – All Keywords Demonstrated

Aim

Aim :- To demonstrate all Java exception handling keywords: try, catch, finally, throw, throws, and multi-catch.

Program (Java Code)

```
public class Program7 {

    static void riskyMethod(int x) throws Exception {
        if (x == 0) throw new ArithmeticException("Cannot divide by zero");
        if (x < 0) throw new IllegalArgumentException("Negative input not allowed");
        System.out.println("Result: " + (100 / x));
    }

    public static void main(String[] args) {

        // try, catch, finally
        System.out.println("--- try / catch / finally ---");
        try {
            riskyMethod(0);
        } catch (ArithmeticException e) {
            System.out.println("Caught ArithmeticException: " + e.getMessage());
        } catch (Exception e) {
            System.out.println("Caught Exception: " + e.getMessage());
        } finally {
            System.out.println("Finally block always executes.");
        }

        // throw
        System.out.println("\n--- throw ---");
        try {
            throw new RuntimeException("Manually thrown exception");
        } catch (RuntimeException e) {
            System.out.println("Caught: " + e.getMessage());
        }

        // throws (declared in method signature above)
        System.out.println("\n--- throws ---");
        try {
            riskyMethod(-5);
        } catch (Exception e) {
            System.out.println("Caught via throws: " + e.getMessage());
        }

        // Multi-catch
        System.out.println("\n--- Multi-catch ---");
        try {
            String s = null;
            s.length();   // NullPointerException
        } catch (NullPointerException | ArrayIndexOutOfBoundsException e) {
            System.out.println("Multi-catch: " + e.getClass().getSimpleName());
        }

        // Nested try-catch
        System.out.println("\n--- Nested try-catch ---");
        try {
            try {
```

```
                int[] arr = new int[3];
                arr[10] = 5;
            } catch (ArrayIndexOutOfBoundsException e) {
                System.out.println("Inner catch: " + e.getMessage());
                throw new RuntimeException("Re-thrown from inner catch");
            }
        } catch (RuntimeException e) {
            System.out.println("Outer catch: " + e.getMessage());
        }

        System.out.println("\nAll exception keywords demonstrated successfully.");
    }
}
```

Output

```
--- try / catch / finally ---
Caught ArithmeticException: Cannot divide by zero
Finally block always executes.
```

```
--- throw ---
Caught: Manually thrown exception
```

```
--- throws ---
Caught via throws: Negative input not allowed
```

```
--- Multi-catch ---
Multi-catch: NullPointerException
```

```
--- Nested try-catch ---
Inner catch: Index 10 out of bounds for length 3
Outer catch: Re-thrown from inner catch
```

```
All exception keywords demonstrated successfully.
```

Result: The program executed successfully and produced the expected output.

Program 8: Bank Account – Custom NoMoneyException

Aim

Aim :- To implement a bank account withdrawal system that throws a custom NoMoneyException on insufficient balance.

Program (Java Code)

```
import java.util.Scanner;

class NoMoneyException extends Exception {
    NoMoneyException(double balance, double amount) {
        super(String.format(
            "Insufficient funds! Tried to withdraw %.2f but balance is only %.2f",
            amount, balance));
    }
}

class BankAccount {
    private String owner;
    private double balance;

    BankAccount(String owner, double initialBalance) {
        this.owner = owner;
        this.balance = initialBalance;
    }

    void withdraw(double amount) throws NoMoneyException {
        if (amount > balance) {
            throw new NoMoneyException(balance, amount);
        }
        balance -= amount;
        System.out.printf("Withdrawn:  %.2f  |  Remaining  balance:  %.2f%n",  amount,
balance);
    }

    void showBalance() {
        System.out.printf("Account: %s | Balance: %.2f%n", owner, balance);
    }
}

public class Program8 {
    public static void main(String[] args) {
        BankAccount account = new BankAccount("Rahul", 5000.00);
        account.showBalance();

        Scanner sc = new Scanner(System.in);
        System.out.print("Enter amount to withdraw: ");
        double amount = sc.nextDouble();

        try {
            account.withdraw(amount);
        } catch (NoMoneyException e) {
            System.out.println("NoMoneyException: " + e.getMessage());
        } finally {
            account.showBalance();
        }
    }
}
```

Output

```
Account: Rahul | Balance: 5000.00
Enter amount to withdraw: 7000.00
NoMoneyException: Insufficient funds! Tried to withdraw 7000.00 but balance is only
5000.00
Account: Rahul | Balance: 5000.00
```

```
--- Second run (successful withdrawal) ---
Account: Rahul | Balance: 5000.00
Enter amount to withdraw: 2000.00
Withdrawn: 2000.00 | Remaining balance: 3000.00
Account: Rahul | Balance: 3000.00
```

Result: The program executed successfully and produced the expected output.

Program 9: Train Platform – One Train at a Time (Synchronization)

Aim

Aim :- To simulate train platform access using thread synchronization so only one train uses the platform at a time.

Program (Java Code)

```
class Platform {
    private String name;
    private boolean occupied = false;

    Platform(String name) { this.name = name; }

    synchronized void usePlatform(String trainName) {
        while (occupied) {
            System.out.println(trainName + " is waiting. Platform is occupied.");
            try      {      wait();      }      catch      (InterruptedException      e)
{ Thread.currentThread().interrupt(); }
        }
        occupied = true;
        System.out.println(trainName + " has entered platform: " + name);
        try     {     Thread.sleep(2000);     }     catch     (InterruptedException     e)
{ Thread.currentThread().interrupt(); }
        System.out.println(trainName + " has left platform: " + name);
        occupied = false;
        notifyAll();
    }
}

class Train extends Thread {
    private Platform platform;

    Train(String name, Platform platform) {
        super(name);
        this.platform = platform;
    }

    public void run() {
        platform.usePlatform(getName());
    }
}

public class Program9 {
    public static void main(String[] args) {
        Platform platform = new Platform("Platform-1");

        Train t1 = new Train("Train-A (Express)", platform);
        Train t2 = new Train("Train-B (Local)",   platform);
        Train t3 = new Train("Train-C (Freight)",  platform);

        t1.start();
        t2.start();
        t3.start();

        try {
            t1.join(); t2.join(); t3.join();
        } catch (InterruptedException e) {
            e.printStackTrace();
```

```
        }
        System.out.println("All trains have used the platform.");
    }
}
```

Output

```
Train-A (Express) has entered platform: Platform-1
Train-B (Local) is waiting. Platform is occupied.
Train-C (Freight) is waiting. Platform is occupied.
Train-A (Express) has left platform: Platform-1
Train-B (Local) has entered platform: Platform-1
Train-C (Freight) is waiting. Platform is occupied.
Train-B (Local) has left platform: Platform-1
Train-C (Freight) has entered platform: Platform-1
Train-C (Freight) has left platform: Platform-1
All trains have used the platform.
```

Result: The program executed successfully and produced the expected output.

Program 10: Restaurant & Waiter – Order Handling with Thread Synchronization

Aim

Aim :- To simulate a restaurant order system where a Waiter and Chef coordinate using thread synchronization.

Program (Java Code)

```
import java.util.LinkedList;
import java.util.Queue;

class Kitchen {
    private Queue<String> orders = new LinkedList<>();
    private boolean chefFree = true;
    private boolean kitchenFree = true;

    synchronized void placeOrder(String order) {
        while (!kitchenFree) {
            System.out.println("Kitchen busy! Waiter waiting to place order: " + order);
            try      {      wait();      }      catch      (InterruptedException      e)
{ Thread.currentThread().interrupt(); }
        }
        kitchenFree = false;
        orders.add(order);
        System.out.println("[Waiter] Order placed: " + order);
        kitchenFree = true;
        notifyAll();
    }

    synchronized void deliverOrder() {
        while (orders.isEmpty()) {
            System.out.println("[Chef] No orders yet. Chef waiting...");
            try      {      wait();      }      catch      (InterruptedException      e)
{ Thread.currentThread().interrupt(); }
        }
        while (!chefFree) {
            System.out.println("[Chef] Chef is busy, waiting...");
            try      {      wait();      }      catch      (InterruptedException      e)
{ Thread.currentThread().interrupt(); }
        }
        chefFree = false;
        String order = orders.poll();
        System.out.println("[Chef] Preparing order: " + order);
        try     {     Thread.sleep(1500);     }     catch     (InterruptedException     e)
{ Thread.currentThread().interrupt(); }
        System.out.println("[Chef] Order delivered: " + order);
        chefFree = true;
        notifyAll();
    }
}

class Waiter extends Thread {
    private Kitchen kitchen;
    private String[] menuOrders;

    Waiter(Kitchen k, String[] orders) {
        super("Waiter");
        this.kitchen = k;
```

```
        this.menuOrders = orders;
    }

    public void run() {
        for (String o : menuOrders) {
            kitchen.placeOrder(o);
            try    {    Thread.sleep(500);    }    catch    (InterruptedException    e)
{ Thread.currentThread().interrupt(); }
        }
    }
}

class Chef extends Thread {
    private Kitchen kitchen;
    private int totalOrders;

    Chef(Kitchen k, int total) {
        super("Chef");
        this.kitchen = k;
        this.totalOrders = total;
    }

    public void run() {
        for (int i = 0; i < totalOrders; i++) {
            kitchen.deliverOrder();
        }
    }
}

public class Program10 {
    public static void main(String[] args) {
        Kitchen kitchen = new Kitchen();
        String[] orders = {"Pasta", "Pizza", "Burger", "Salad", "Soup"};

        Waiter waiter = new Waiter(kitchen, orders);
        Chef   chef   = new Chef(kitchen, orders.length);

        waiter.start();
        chef.start();

        try {
            waiter.join();
            chef.join();
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        System.out.println("\nAll orders have been served. Restaurant closing.");
    }
}
```

Output

```
[Waiter] Order placed: Pasta
[Chef] Preparing order: Pasta
[Waiter] Order placed: Pizza
[Chef] Order delivered: Pasta
[Chef] Preparing order: Pizza
[Waiter] Order placed: Burger
[Chef] Order delivered: Pizza
[Chef] Preparing order: Burger
[Waiter] Order placed: Salad
[Chef] Order delivered: Burger
[Chef] Preparing order: Salad
[Waiter] Order placed: Soup
[Chef] Order delivered: Salad
[Chef] Preparing order: Soup
```

```
[Chef] Order delivered: Soup
```

```
All orders have been served. Restaurant closing.
```

Result: The program executed successfully and produced the expected output.